

Inside IIM Assignment

Date 9/7

AGENT STEERING BLUEPRINT: AEGIS AI (V5 — FINAL)

****Role:**** Senior Full-Stack & AI Engineer

****Objective:**** Build a production-grade, evidence-first, multi-agent investment research platform incrementally.

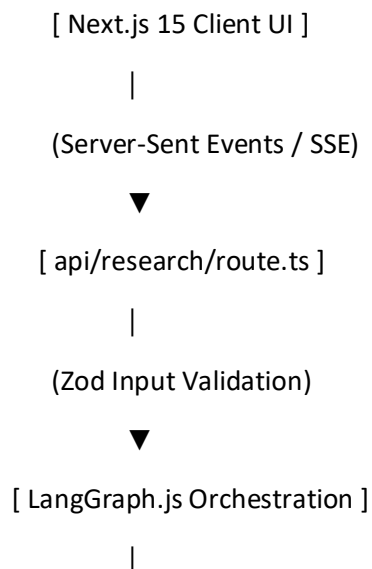
****Execution Mode:**** Strictly sequential. Finish, debug, log, and commit each numbered step before moving to the next. Do not reorder, merge, or skip steps.

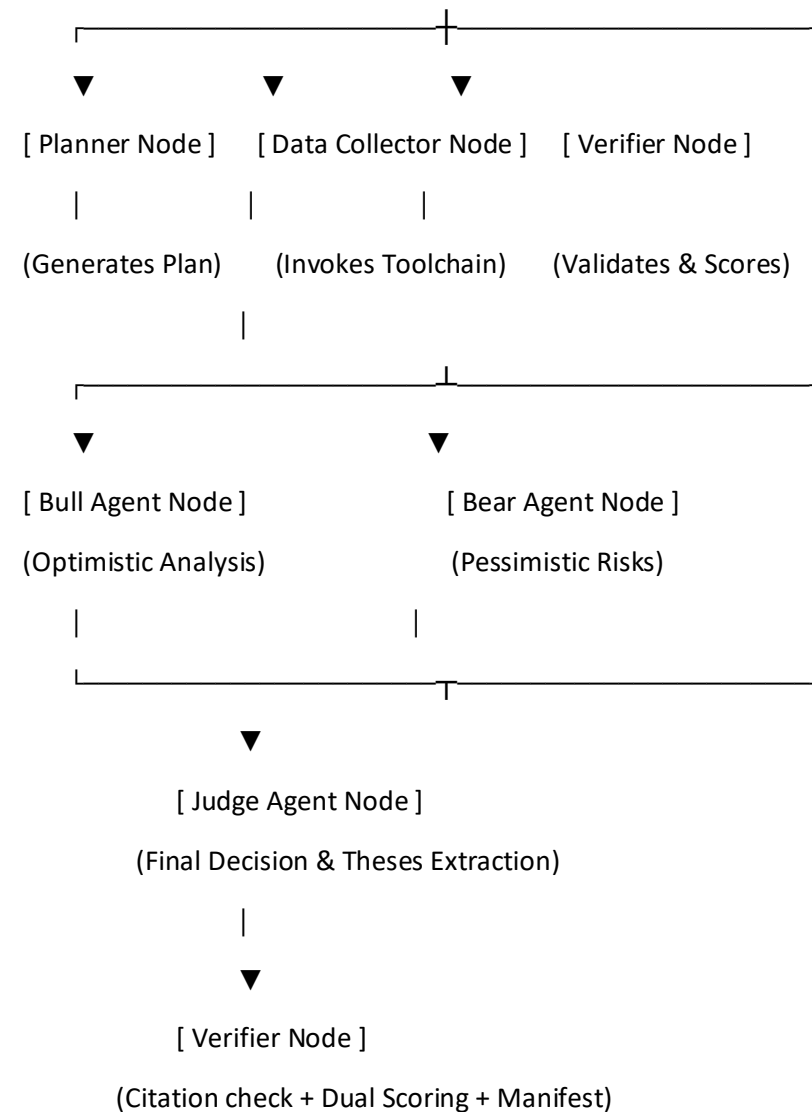
****Product philosophy (this sentence governs every decision below):****

> Every investment recommendation must be backed by verifiable evidence before an AI is allowed to recommend an action. The AI never decides — it synthesizes evidence and explains a decision that a deterministic scoring engine already computed.

1. Core System Architecture

...





...

****Naming note (documentation-only, do not rename files):**** In README.md, DECISIONS.md, and code comments, refer to the Judge conceptually as the ****Decision Engine****, the ``tools/`` wrappers as ****Evidence Providers****, and the cache as the ****Evidence Store****. File and folder names stay exactly as listed in Section 2 — renaming physical files mid-build creates import churn with no functional benefit. Vocabulary lives in prose, not paths.

2. Definitive Directory Structure

Maintain this exact layout. Do not add extraneous files or directories unless a step below explicitly calls for it.

...

aegis-ai/

- |— README.md
- |— DECISIONS.md
- |— ai-logs/
 - | |— 001-project-setup.md
 - | |— 002-domain-types-provenance.md
 - | |— 003-input-security-logging.md
 - | |— 004-landing-interface.md
 - | |— 005-evidence-providers-caching.md
 - | |— 006-langgraph-scaffolding.md
 - | |— 007-bull-bear-agents.md
 - | |— 008-judge-decision-engine.md
 - | |— 009-verifier-dual-scoring.md
 - | |— 010-graph-integration-test.md
 - | |— 011-streaming-api-route.md
 - | |— 012-dashboard-ui.md
 - | |— 013-testing-health-check.md
 - | |— 014-documentation.md
 - | |— 015-deployment.md
- |— .env.example
- |— .gitignore
- |— next.config.js

```
└─ package.json
└─ src/
  │ └─ app/
  │ │ └─ page.tsx
  │ │ └─ research/[company]/page.tsx
  │ │ └─ api/
  │ │ │ └─ research/route.ts
  │ │ │ └─ health/route.ts
  │ │ └─ layout.tsx
  │ └─ components/
  │ │ └─ ui/
  │ │ └─ research/
  │ │ └─ landing/
  │ └─ agents/
  │ │ └─ graph.ts
  │ │ └─ nodes/
  │ │ │ └─ planner.ts
  │ │ │ └─ dataCollector.ts
  │ │ │ └─ bullAgent.ts
  │ │ │ └─ bearAgent.ts
  │ │ │ └─ judgeAgent.ts
  │ │ │ └─ verifier.ts
  │ │ └─ prompts/
  │ │ └─ schemas/
  │ └─ tools/
  │ │ └─ financialData.ts
  │ │ └─ newsSearch.ts
  │ │ └─ secFilings.ts
  └─ lib/
```

```

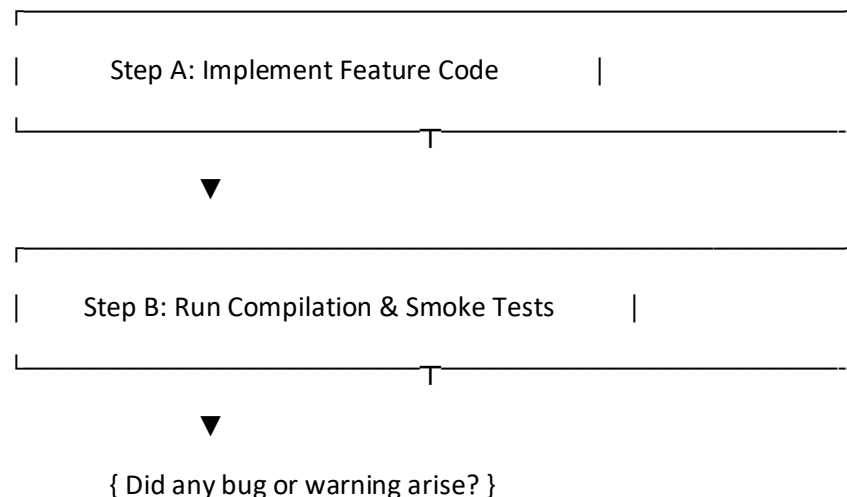
| | └─ cache.ts
| | └─ rateLimit.ts
| | └─ logger.ts
| | └─ validation.ts
| └─ types/
|   └─ domain.ts
└─ middleware.ts
└─ tests/
    └─ unit/
    └─ integration/
...

```

3. Strict Execution Protocol

For every numbered step in Section 4, follow this loop exactly. Do not skip steps, and do not batch multiple steps into one commit.

...



└─ Yes → [Fix and re-test]

└─ No



```
| Step C: Document to ai-logs/[NNN-slug].md |
| Record: exact prompt sent, what was kept, what was |
| changed/rejected and why. Update DECISIONS.md if |
| this step involved any trade-off. |
└──────────────────────────────────────────┘
```



```
| Step D: Stage and commit using Conventional Commits |
| (feat: / fix: / chore: / docs: / test: / refactor:) |
└──────────────────────────────────────────┘
```



```
| Step E: Move to next numbered step |
└──────────────────────────────────────────┘
```

...

4. Phased Step-by-Step Build Plan

Phase 1 — Environment, Guardrails & Domain

Step 001: Project Setup

- Initialize Next.js 15 with TypeScript, Tailwind CSS, ESLint, Prettier.

- Configure baseline security headers in `src/middleware.ts` (XSS protection, CSP scaffolding, strict CORS).
- Create `.env.example`; add `.env` to `.gitignore`.

****Step 002: Domain Types & Provenance****

- Create `src/types/domain.ts`. Define interfaces: `Company`, `Evidence`, `Citation`, `InvestmentThesis`, `RiskFactor`, and `ResearchManifest` (fields: `requestId`, `timestamp`, `sourcesUsed`, `apisCalled`, `cached`, `retries`, `failures`, `verifiedCitations`, `researchDurationMs`).
- Define the universal tool output envelope type: `{ value: unknown, source: string, fetchedAt: string, isMock: boolean }`.

****Step 003: Base Input, Security & Logging****

- Set up structured logging (Pino or Winston) in `src/lib/logger.ts`, with a request/correlation ID attached to every log line.
- Implement an in-memory token-bucket rate limiter in `src/lib/rateLimit.ts`.
- Write Zod schemas in `src/lib/validation.ts` to sanitize incoming payloads (company name: alphanumeric + limited punctuation, max length) — this is also the first line of defense against prompt-injection-shaped input.

Phase 2 — Structural Foundation & Tools

****Step 004: Landing Interface****

- Build a minimalist search interface (Shadcn + Tailwind) on `src/app/page.tsx`.
- Client-side validation must mirror the server Zod schema exactly (same constraints, both places).

****Step 005: Evidence Providers & Caching****

- Create `src/lib/cache.ts`: simple in-memory LRU cache.
- Build `src/tools/financialData.ts`, `newsSearch.ts`, `secFilings.ts`. Every external call wrapped with an explicit 8-second timeout and a fallback path.
- ****CRITICAL:**** every tool return must use the provenance envelope from Step 002. If an API quota is exhausted or a call fails after retry, return fallback/sample data with `isMock: true` set explicitly — never silently.

Phase 3 — Multi-Agent Logic Engine

****Step 006: LangGraph Scaffolding****

- Create `src/agents/graph.ts`. Instantiate the state graph using the `domain.ts` types as the state shape. Add a `Planner` node and one temporary dummy execution node purely to prove state transitions work end to end.
- Note in the log file: this dummy node is scaffolding and will be removed in Step 010.

****Step 007: Bull & Bear Agents****

- Implement `src/agents/nodes/bullAgent.ts` and `bearAgent.ts`.
- Strict system prompts: the Bull agent must ONLY surface growth vectors and optimistic signals; the Bear agent must ONLY surface liabilities, structural risk, and negative sentiment. Neither agent should hedge toward the middle — that defeats the purpose of running both.

****Step 008: Judge (Decision Engine) & Structured Extraction****

- Build `src/agents/nodes/judgeAgent.ts` using LangChain's structured-output binding against a Zod schema.
- Output fields: `recommendation` (BUY / PASS), `investmentTheses` (string array), `sourceCitations` (array mapping directly to tool output indices).
- ****The Judge does not output a confidence score.**** Confidence is computed downstream in Step 009, never invented by the LLM.

****Step 009: Verifier Node & Dual-Scoring Engine****

- Build `src/agents/nodes/verifier.ts`. Cross-check every entry in the Judge's `sourceCitations` against the actual evidence stored in graph state. Unverifiable citations get logged to a `verificationWarnings` array in state, not silently dropped.
- Compute two separate deterministic scores from real graph-state signals (never from an LLM guess):
 1. ****`researchQuality`**** (0–100): a function of source coverage, data freshness / non-mock ratio, and number of sources successfully verified.
 2. ****`decisionConfidence`**** (0–100): a function of Bull/Bear agreement, evidence strength behind the recommendation, and any contradictions found.

- Assemble the `ResearchManifest` (Step 002 type) with real values — request ID, timestamps, retries, failures, verified-citation count, both scores — and attach it to final graph state. This manifest is returned to the client as the "receipt" for the recommendation, not just logged server-side.

Phase 4 — Integration & Streaming

****Step 010: Full Graph Integration Test (No UI)****

- Wire the complete path: Planner → Data Collector → Bull/Bear → Judge → Verifier in `src/agents/graph.ts`.
- ****Remove the dummy node and any Step 006 scaffolding now**** — it has served its purpose.
- Write a local script (`scripts/test-graph.ts` or similar, run via `tsx`/`ts-node`) that executes the graph against a hardcoded real company name and prints the final state to the terminal. Debug all state-shape and transition bugs here, before the API route or UI exist. This is the highest-leverage debugging step in the whole plan — do not shortcut it.

****Step 011: Streaming API Route****

- Configure `src/app/api/research/route.ts` to accept the validated POST payload, run the LangGraph sequence, and stream state updates to the client via Server-Sent Events.

****Step 012: Dashboard UI & Citations Canvas****

- Build `src/app/research/[company]/page.tsx` with a Framer Motion animated timeline reflecting real graph-state transitions (not a fake canned animation — it should reflect actual node progress from the SSE stream).
- ****Mandatory rendering order, top to bottom:**** Evidence table → Verification / Manifest receipt → Agent reasoning (Bull vs. Bear) → Final recommendation. The recommendation is the conclusion the user reaches, not the headline they see first.
- ****Mandatory fallback disclosure:**** any data point where `isMock === true` must show a visible "Using fallback/sample data" badge next to that specific metric. Never present mock data identically to live data.

Phase 5 — Testing, Documentation & Deployment

****Step 013: Testing & Health Check****

- Implement ``src/app/api/health/route.ts`` for uptime/health checks.
- Populate ``tests/unit`` and ``tests/integration`` covering, at minimum: Zod validation edge cases, the dual-scoring engine's math (given known inputs, assert known outputs), and tool fallback triggering correctly under simulated API failure.

****Step 014: Documentation Finalization****

- Finalize ``README.md`` with the sections the assignment requires: Overview, How to run it, How it works, Key decisions & trade-offs, Example runs, What you would improve with more time.
- ****Example runs must include actual output****, not placeholders — run the deployed or local app against 2–3 real companies and paste the real recommendation, scores, and manifest. ****At least one example run must show the fallback path in action**** (an ``isMock: true`` badge visible), proving the graceful-degradation claim is real rather than aspirational.
- In "How it works," explicitly document: the dual-verification points (evidence intake via the provenance envelope, and citation-output via the Verifier), the `researchQuality/decisionConfidence` split and why they're kept separate, and that LLM-provider swapping is handled by LangChain's model abstraction (no custom interface needed).
- Update ``DECISIONS.md`` with a final pass covering anything not already logged incrementally.

****Step 015: Deployment****

- Deploy to Vercel. Set all environment variables in the Vercel dashboard (never commit them). Run live smoke tests confirming the 8-second tool timeouts stay within Vercel's serverless function limits.

5. Security & Stability Requirements (apply throughout, not just at the relevant step)

- ****No client leaks:**** all API keys stay server-side. Never use a ``NEXT_PUBLIC_`` prefix for any key or secret.
- ****Strict LLM output typing:**** every node that calls an external model validates the response against its Zod schema before writing to graph state. A schema failure is caught and logged, not allowed to silently corrupt state.
- ****Graceful degrade:**** any unhandled exception in an agent node is caught, logged with its request ID, and routed to a fallback path — it must never lock the SSE stream or crash the request.

- ****Input sanitization first:**** validation happens before the payload ever reaches the graph, not as an afterthought inside a node.

6. Definition of Done

A step is not complete until all of the following are true:

1. Code compiles with no TypeScript errors and no ESLint warnings.
2. The relevant `ai-logs/NNN-slug.md` file exists with the real prompt used.
3. `DECISIONS.md` reflects any trade-off made in that step.
4. A single Conventional Commit exists for that step (squash first if there are "wip"/fix-typo commits along the way).
5. The feature actually runs — a step is not "done" because the code compiles; run it and confirm the behavior described in Section 4 before moving on.